

MinZ vs Nanz — Feature Gap Analysis

MinZ Project

2026-03-13

Contents

MinZ vs Nanz — Feature Gap Analysis & Resolution Proposal	1
Executive Summary	1
Feature Comparison Matrix	2
Gap Details & Resolution Proposals	3
1. Enums — Priority: HIGH	3
2. @error Propagation — Priority: HIGH	4
3. Import System — Priority: HIGH	5
4. Type Aliases — Priority: MEDIUM	5
5. String Literals — Priority: MEDIUM	5
6. @define Macros — Priority: LOW	6
7. @if/@elif/@else Conditional Compilation — Priority: LOW	6
8. module Declaration — Priority: LOW	7
9. @minz[[[...]]] / @lua[[[...]]] — Priority: DEFERRED	7
10. Pattern Matching — Priority: DEFERRED	7
Implementation Roadmap	8
Phase 1: Language Essentials (1-2 days)	8
Phase 2: Error Handling + Modules (2-3 days)	8
Phase 3: Metaprogramming (optional, 1-2 days)	8
Phase 4: Deferred	8
What Nanz Has That MinZ Never Did	8
Conclusion	9

MinZ vs Nanz — Feature Gap Analysis & Resolution Proposal

Date: 2026-03-13 **Status:** Reference document for Nanz feature parity roadmap

Executive Summary

Nanz (.nanz) is the **active frontend** of the MinZ compiler, targeting the MIR2 backend.
MinZ (.minz) is the **frozen frontend**, targeting MIR1 + old codegen.

Nanz already has **all core language features** and several features MinZ never had (PFCCO, PreallocCoalesce, dual-VM asserts, as cast, mzn native backend, signed comparison, 6502 backend). However, MinZ has **metaprogramming features** and a **module/import system** that Nanz lacks.

This document catalogs every gap and proposes how to close each one.

Feature Comparison Matrix

Feature	MinZ	Nanz	Gap?
Core types (u8, u16, i8, i16, bool, void)	Yes	Yes	—
u24, u32, i32	Partial	Yes	—
Fixed-point (f8.8, etc.)	No	Yes (parsed)	Nanz ahead
Functions (fun/fn, overloading)	Yes	Yes	—
Multiple return values	Yes	Yes	—
Structs + field access	Yes	Yes	—
Struct literals	Yes	Yes	—
Interfaces (zero-cost)	Yes	Yes	—
UFCS	Yes	Yes	—
Operator overloading	Yes	Yes	—
Lambdas (zero-cost)	Yes	Yes	—
Iterator chains	Yes	Yes	—
Pointers, arrays, indexing	Yes	Yes	—
Control flow (if/while/for/switch/break/continue)	Yes	Yes	—
Globals (with at ())	Yes	Yes	—
@extern (with RST opt)	Yes	Yes	—
@smc parameters	Partial	Yes	Nanz ahead
Compile-time assertions	No	Yes	Nanz ahead
as cast syntax	No	Yes	Nanz ahead
Signed comparison (i8/i16)	No	Yes	Nanz ahead
mzn native backend (C99+QBE)	No	Yes	Nanz ahead
6502 backend	No	Yes	Nanz ahead
PreallocCoalesce	No	Yes	Nanz ahead
PFCCO contracts	No	Yes	Nanz ahead
—	—	—	—
Enums	Yes	No	Gap
String literals	Yes	No	Gap

Feature	MinZ	Nanz	Gap?
String interpolation ("{expr}")	Yes	No	Gap
@error propagation	Yes	No	Gap
@define macros	Yes	No	Gap
@if/@elif/@else conditionals	Yes	No	Gap
@minz[[[...]]]	Partial	No	Gap
@lua[[[...]]]	Aspirational	No	Not real
import system	Yes	No	Gap
module declaration	Yes	No	Gap
Type aliases (type X = Y)	Yes	No	Gap
Pattern matching	Partial	No	Gap (both incomplete)
Generators (gen/yield)	Planned	No	Both missing

Gap Details & Resolution Proposals

1. Enums — Priority: HIGH

MinZ syntax:

```
enum State { IDLE, RUNNING, PAUSED }
global state: State = State::IDLE

fun is_active(s: State) -> bool {
    return s == State::RUNNING
}
```

What MinZ does: Parser creates EnumDecl AST node. Semantic analyzer assigns integer values (0, 1, 2...) to variants. Codegen treats enum values as integer constants.

Resolution for Nanz:

```
enum State { IDLE, RUNNING, PAUSED } // auto-numbered: 0, 1, 2
enum Color { RED = 1, GREEN = 2, BLUE = 4 } // explicit values

global state: State = State::IDLE
```

Implementation plan: 1. Add tokEnum to lexer (trivial — copy tokStruct pattern) 2. Add parseEnumDecl() → stores variant names + values in enumDefs map[string][]enumVariant 3. In expression parsing, recognize EnumName::Variant → IntLitExpr with the variant's integer value 4. Enum type resolves to u8 (≤256 variants) or u16 (>256) 5. No HIR changes needed — enums are constants by the time they reach HIR

Effort: ~100 LOC in parse.go. Half a day.

Why it matters: Enums are a fundamental language feature. State machines, command types, error codes — all are cleaner with enums than magic numbers. Every showcase game (Tetris, etc.) will want them.

2. @error Propagation — Priority: HIGH

MinZ syntax:

```
fun safe_divide?(a: u8, b: u8) -> u8 ! DivError {
    if b == 0 { @error(DivError.DivByZero) }
    return a / b
}

fun caller() -> u8 {
    let result = safe_divide?(10, x) // CY set on error
    // ... use result ...
}
```

What MinZ does: ? suffix marks fallible function. ! ErrorType declares the error enum. @error(variant) sets CY flag + A = error code, then returns. Caller checks CY.

Resolution for Nanz:

```
enum MathError { DivByZero, Overflow }

fun safe_divide(a: u8, b: u8) -> u8 ! MathError {
    if b == 0 { return error MathError::DivByZero }
    return a / b
}
```

Implementation plan: 1. Depends on: Enums (item 1) 2. Add ! ErrorType to parseFunDecl return type parsing 3. Add return error expr statement variant → sets CY + A and returns 4. In HIR: FallibleFunc flag + ErrorType field on Func 5. In MIR2 lowering: return error X → LD A, X / SCF / RET 6. Caller: after CALL, JR C, .error_handler 7. Z80-native: CY flag is the cheapest possible error signal (0 bytes, 0T to set via SCF)

Effort: ~150 LOC. Depends on enums. One day.

Why it matters: Error propagation via CY flag is the Z80's natural error mechanism. ROM routines use it. CP/M uses it. This makes Nanz programs interoperate cleanly with system code.

3. Import System — Priority: HIGH

MinZ syntax:

```
import stdlib.math.fast;
import stdlib.graphics.screen;
```

Resolution for Nanz:

```
import math // looks for math.nanz in search path
import stdlib.graphics.screen // stdlib/graphics/screen.nanz
```

Implementation plan: 1. Add `parseImportDecl()` — import dotted.path at top of file 2. Search paths: current dir, stdlib/, configurable `-I` flag 3. Parse imported file → get `*hir.Module` 4. Merge into main module via existing `hir.MergeModules()` (already exists for PL/M multi-file) 5. Symbol collision: error if two modules define same function name (no shadowing) 6. Circular import detection: track import stack, error on cycle

Effort: ~200 LOC. Two days (most effort in path resolution + testing).

Why it matters: Without imports, every Nanz program is a single file. This blocks stdlib usage, code reuse, and any project larger than a demo. The game roadmap (Tetris) needs this for separating game logic from platform code.

4. Type Aliases — Priority: MEDIUM

MinZ syntax:

```
type Byte = u8
type Addr = u16
type Color = u8
```

Resolution for Nanz:

```
type Addr = u16
type Color = u8
```

Implementation plan: 1. Add `parseTypeAlias()` — type NAME = type at top level 2. Store in `typeAliases` `map[string]mir2.Ty` on parser 3. In type resolution (`resolveType`), check `typeAliases` before struct lookup 4. Pure syntactic sugar — aliases are transparent, no runtime representation

Effort: ~30 LOC. One hour.

5. String Literals — Priority: MEDIUM

Current state: Nanz lexer tokenizes strings but they're only used in `@print` meta-functions. No string type in MIR2.

Resolution for Nanz:

```
@extern(0x0010) fun PRINT_CHAR(c: u8) -> void

fun print_str(s: ^u8) {
    var i: u8 = 0
    while s[i] != 0 {
        PRINT_CHAR(s[i])
        i = i + 1
    }
}

global greeting: [u8; 6] = "Hello" // null-terminated byte array
```

Implementation plan: 1. String literal "text" → [u8; len+1] array initializer with null terminator 2. In global init: global msg: [u8; N] = "text" → DB "Hello", 0 3. In expressions: string literal → address of anonymous global 4. No string type — strings are ^u8 (C-style). This is honest for Z80.

Effort: ~80 LOC. Half a day.

Why not string interpolation? Interpolation requires runtime formatting (itoa, concatenation, buffer management). On Z80 with no allocator, this is impractical as a language primitive. Better to provide @print_dec / @print_hex metafunctions for output and leave string manipulation to explicit library functions.

6. @define Macros — Priority: LOW

MinZ syntax:

```
@define("fun {0}(x: u8) -> u8 { return x + {1} }", "add_five", "5")
// Expands to: fun add_five(x: u8) -> u8 { return x + 5 }
```

Resolution: Preprocessor pass before lexing.

Implementation plan: 1. Scan source for @define(...) directives 2. Store template + args 3. Expand all @define calls (text substitution with {0}, {1} placeholders) 4. Feed expanded source to existing parser

Effort: ~100 LOC for a simple text preprocessor. Half a day.

Why low priority: Nanz's function system + operator overloading + iterator chains already provide most of the abstraction that @define was used for in MinZ. The remaining use cases (boilerplate code generation) are rare in Z80 programs.

7. @if/@elif/@else Conditional Compilation — Priority: LOW

MinZ syntax:

```
@if(SPECTRUM) {
    fun clear_screen() { ... }    // ZX Spectrum version
} @elif(CPM) {
    fun clear_screen() { ... }    // CP/M version
}
```

Resolution: Target-gated blocks.

Implementation plan: 1. Add @if(CONDITION) parsing at top level 2. Conditions: SPECTRUM, CPM, AGON, GENERIC — derived from --target flag 3. Parser includes/excludes blocks based on condition 4. Can reuse existing asm(z80) / asm(mir2) target-gating pattern

Effort: ~80 LOC. Half a day.

Why low priority: Most Nanz programs target one platform. Cross-platform code can use the import system (import platform-specific modules) instead of conditional compilation.

8. module Declaration — Priority: LOW

```
module game.tetris    // optional, informational
```

Implementation: Add optional module dotted.name at start of file. Store in hir.Module.Name. Used for symbol scoping when imports land.

Effort: ~20 LOC. Trivial.

9. @minz[[[...]]] / @lua[[[...]]] — Priority: DEFERRED

Assessment: These are compile-time code generation features. @minz requires a Go-based execution environment for @emit(). @lua requires embedding a Lua interpreter.

Recommendation: Defer until a real use case demands it. The current Nanz feature set (LUTGen, compile-time asserts, @smc, operator overloading) covers the main code-generation scenarios that these features addressed in MinZ. If metaprogramming is needed, prefer @define (item 6) which is simpler and sufficient for most cases.

10. Pattern Matching — Priority: DEFERRED

Assessment: Neither MinZ nor Nanz has complete pattern matching. MinZ parses the syntax but codegen is partial. Adding full pattern matching (enum destructuring, guard expressions, exhaustiveness checking) is a significant effort (~500+ LOC) with limited Z80 benefit.

Recommendation: Defer. switch + enums covers the practical cases. Revisit when the language targets modern CPUs via QBE/LLVM where pattern matching compiles to jump tables efficiently.

Implementation Roadmap

Phase 1: Language Essentials (1-2 days)

1. **Enums** — fundamental type, blocks @error
2. **Type aliases** — trivial, improves readability
3. **String literals** — needed for any I/O program

Phase 2: Error Handling + Modules (2-3 days)

4. **@error propagation** — depends on enums
5. **Import system** — enables multi-file projects and stdlib

Phase 3: Metaprogramming (optional, 1-2 days)

6. **@define macros** — if needed
7. **@if conditional compilation** — if multi-platform needed

Phase 4: Deferred

8. module declaration (trivial, do whenever)
 9. **@minz[[[...]]]** (wait for demand)
 10. Pattern matching (wait for modern target work)
-

What Nanz Has That MinZ Never Did

These features exist **only in Nanz** and represent the MIR2 backend's advantages:

Feature	Impact
PFCCO interprocedural contracts	Custom calling conventions per function
PreallocCoalesce	Block-param register unification
Dual-VM compile-time asserts	Catches both algorithm and codegen bugs
as cast syntax	expr as u8 alongside u8(expr)
mzn native backend	Compile Nanz → AMD64 via C99 or QBE
Signed comparison (i8/i16)	VM sign-extends for correct signed CmpLt/Le/Gt/Ge
6502 backend	35/35 tests, dual-VM oracle
Trivial inliner	swap(a, b).1 == a → zero instructions

Feature	Impact
ForEachEdge visitor	Clean term edge iteration, ~75 LOC removed
@smc parameters	Baked immediates in instruction stream
LUTGen	Pure functions → compile-time lookup tables
BranchEquiv	VM-proved dead branch elimination
CondRetSink + CmpSubCarry	abs_diff → 4 instructions

Conclusion

The gap between MinZ and Nanz is primarily in **metaprogramming** (macros, conditional compilation) and **module system** (imports). The core language is at parity or beyond. Enums are the single biggest missing feature that affects everyday programming.

The recommended path: **Enums → @error → Imports → Strings**. This gives Nanz everything needed for real programs (games, utilities, system tools) while keeping the language simple and honest about its Z80 target.

MinZ Compiler — modern abstractions, vintage iron, zero overhead.